

Aero 5553 - Advanced Control Theory

Course Notes

Eric A. Mehiel, Ph.D.

Table of contents

Preface	5
1 Dynamic System Control	6
1.1 Slide 2 - Motivation and Learning Outcomes	6
1.1.1 Motivation	6
1.1.2 Learning Outcomes	6
1.2 Slide 3 - The ‘mind’ of a controls engineer	6
1.3 Slide 4 - What is feedback?	6
1.4 Slide 5 - The Centrifugal Governor	7
1.5 Slide 6 - Steam Engine as a Block Diagram	7
1.6 Slide 7 - Cruise Control	7
1.7 Slide 8 - Biological and Ecological Systems	7
1.8 Slide 9 - Feedback Properties	8
1.9 Slide 10 - Some Notes	8
1.10 Slide 11 - PID: An Intuitive Approach for Linear Systems	8
1.11 Slide 12 - Buckaroo Banzi	8
1.12 Slide 13 - Control Theory Mind Map	8
2 Classical Control Highlights	9
2.1 Slide 2 - Motivation and Learning Outcomes	9
2.1.1 Motivation	9
2.1.2 Learning Outcomes	9
2.2 Slide 3 - Functions of a Complex Variable	9
2.3 Slide 4 - Poles and Zeros Examples	10
2.4 Slide 5 - The Laplace Transform	11
2.5 Slide 6 - Properties of the Laplace Transform	11
2.6 Slide 7 - Useful Theorems	12
2.7 Slide 8 - Why Laplace?	12
2.7.1 Example - Mass-Spring-Damper System (MSD)	13
2.8 Slide 9 - Another look at the Transfer Function	15
2.9 Slide 10 - Step Response	16
2.10 Slide 11 - Performance of the MSD Step Response	20
2.11 Slide 12 - Performance Metrics	20
2.12 Slide 13 - Performance and Stability with Feedback	20
2.12.1 Proportional Control	20

2.13	Slide 14 - Proportional Feedback	21
2.14	Slide 15 - Final Value Theorem	21
2.15	Slide 16 - Proportional + Derivative Feedback	22
2.16	Slide 17 - Full PID Feedback	23
3	System Modeling	24
3.1	Slide 2 - Motivation and Learning Outcomes	24
3.1.1	Developing a models is step <i>ZERO</i>	24
3.2	Slide 3 - The Heritage of Mechanics	24
3.3	Slide 4 - State, Time, and Space	24
3.4	Slide 5 - The Heritage of Electrical Engineering	25
3.5	Slide 6 - Step and Frequency Response	25
3.6	Slide 7 - The Control View - State Space	26
3.7	Slide 8 - Summary	26
4	State Space Modeling	27
4.1	Slide 2 - Motivation and Learning Outcomes	27
4.1.1	The Fundamental Abstraction	27
4.2	Slide 3 - Start with ODEs	27
4.3	Slide 4 - For LTI systems	27
4.4	Slide 7 - Example - The Inverted Pendulum	28
4.5	Slide 8 - State Space Model of the Inverted Pendulum	28
4.6	Slide 9 - Linearization of the Inverted Pendulum	29
4.7	Slide 10 - Simulink Example of the Inverted Pendulum	29
4.8	Slide 11 - Block Diagrams Overview	29
4.9	Slide 12 - Summary	29
4.9.1	Developing a model is step <i>ZERO</i> in the design of a control system. . .	29
5	Dynamic Behavior - Stability	30
5.1	Motivation	30
5.2	Slide 3 - Solving	30
5.3	Slide 4 - Examples - Finite Escape Time	30
5.4	Slide 5 - Non-uniqueness of Solutions	31
5.5	Slide 6 - Lipschitz Continuity	31
5.6	Slide 7 - Examples of Lipschitz Continuity	31
5.7	Slide 8 - Qualitative Analysis - Phase Portraits	32
5.8	Slide 9 - Equilibrium Points	32
5.9	Slide 10 - Examples	32
5.10	Slide 11 - Summary	33
6	Linear Systems	34
6.1	Modal Example - 2 MSD system	34
6.1.1	Modal Decomposition Process	35

7	Root Locus	36
7.1	Learning Objectives	36
8	Frequency Response	37
8.1	Learning Objectives	37
9	PID Design	38
9.1	Learning Objectives	38
	Appendices	39
A	Matrices and Vector Spaces: A Compact Reference	39
A.1	The unifying idea	39
A.2	Vector spaces, span, and dimension	40
A.3	Matrices and their basic operations	40
A.3.1	Block matrices	41
A.4	Determinant, rank, invertibility, and degeneracy	41
A.5	Trace, similarity, and diagonalization	42
A.6	Inner products, outer products, and norms	43
A.7	Singular value decomposition and conditioning	44
A.8	Quadratic forms and definite matrices	45
A.9	How the concepts fit together	46
A.10	Source links	46

Preface

HERE IS AN UPDATE - for real

These notes accompany a graduate course in Advanced Control Systems at Cal Poly, San Luis Obispo. The course is taught by Dr. E. Mehiel and is based on several books. Primarily, the course is based on the book,

- “Feedback Systems: An Introduction for Scientists and Engineers” by Karl J. Åström and Richard M. Murray, Princeton University Press, Version v3.1.5 (2020-07-24), [Here is a pdf](#)
- “Linear System Theory: Second Edition” by J. Hespanha, Princeton University Press, 2018,
- “Robust and Adaptive Control: With Aerospace Applications” by E. Lavretsky and K. Wise, Springer, 2013.

The course is designed to provide students with a solid foundation in modern control theory and its applications.

Each chapter is the set of notes I use when presenting to the class and contains:

- Motivation
- Theory
- Worked examples
- Summary

1 Dynamic System Control

1.1 Slide 2 - Motivation and Learning Outcomes

1.1.1 Motivation

- Control is just “making things work.”
- From a physics perspective the difference between an uncontrolled and controlled dynamic system is irrelevant
- From an engineering perspective, the difference between the two is central to design and performance
- But the mathematical techniques are the same, so, it’s theory...
 - Control theory is the mathematical formalism we use to analyze and design (engineer) dynamic systems (things that move).
- https://en.wikipedia.org/wiki/Control_theory

1.1.2 Learning Outcomes

- Look inside the ‘mind’ of a controls engineer
- Discuss important concepts, like feedback, robustness, and Automation
- Develop a conceptual understanding of PID control

1.2 Slide 3 - The ‘mind’ of a controls engineer

1.3 Slide 4 - What is feedback?

When we develop models of dynamic systems with differential equations, other math and physics, we take a reductive approach. When two or more dynamical systems are connected in a way where both systems depend on output from the other system for input, the dynamical systems are in *feedback*.

Another way of thinking about feedback is that the connected systems cannot be solved independently from each other. The systems are *coupled* and must be solved together.

1.4 Slide 5 - The Centrifugal Governor

An early example of feedback is the centrifugal governor, which was used to control the speed of steam engines. The governor uses feedback to adjust the fuel supply to the engine based on its speed, maintaining a desired speed despite changes in load.

1.5 Slide 6 - Steam Engine as a Block Diagram

Block diagrams are a way to represent the relationships between different components of a system. The steam engine can be represented as a block diagram, with the governor and engine as separate blocks that interact through feedback.

This is a reductive approach to modeling the system which it allows us to analyze and design the system more easily.

Early object oriented programming was based on this idea of modeling systems as blocks that interact with each other. Modelica is a modern programming language that uses this approach to model complex systems.

1.6 Slide 7 - Cruise Control

Cruise control is a common example of feedback control in modern vehicles. The system uses feedback from the vehicle's speed to adjust the throttle and maintain a desired speed, even when going up or down hills.

Cruise control systems started as mechanical systems, but have since evolved into computer-controlled systems that use sensors and algorithms to optimize performance and fuel efficiency.

1.7 Slide 8 - Biological and Ecological Systems

Control theory is not limited to mechanical or electrical systems. Biological and ecological systems also exhibit feedback control, such as the complex chemical systems of cells and how birds fly in formation.

With the advent of mechanical systems and now computer systems, we can design or program a feedback loop.

These days control is a mix of design and computer control with embedded or other computer systems.

1.8 Slide 9 - Feedback Properties

- Robustness – As mass increases, the car still gets to the desired speed in roughly the same amount of time but with different “performance”.
 - Question how much fuel is used? So the cruise control is robust, but there is no consideration of energy.
- Design of Dynamics - Use feedback control to change the closed-loop dynamics of the system. Design a vehicle for some set of requirements, and design a controller to meet performance metrics. Example, the Wright Flyer.
- Automation - Combining control theory and dynamics with logic, heuristics, or AI to achieve more complicated control objectives or enhanced performance or robustness.

1.9 Slide 10 - Some Notes

1.10 Slide 11 - PID: An Intuitive Approach for Linear Systems

- Proportional - Control for right now
- Derivative - Control for the expected future
- Integral - Control for the past
- PID - a blend of all three time lines - this works because of Newton's Laws. How you ended up in your current state is a function of where you were (I), the decision you make right now (P), and where you want to go (D).

1.11 Slide 12 - Buckaroo Banzai

“No matter how much you push the envelope, it'll still be stationary.” - Buckaroo Banzai

“No matter where you go, there you are.” - Buckaroo Banzai

1.12 Slide 13 - Control Theory Mind Map

Where are we going in this class?

2 Classical Control Highlights

2.1 Slide 2 - Motivation and Learning Outcomes

2.1.1 Motivation

- Proportional, Integral, Derivative Control
- For “single-input, single-output” systems, PID is often sufficient
- PID has been around a long time and is well understood
- PID is relatively easy to tune with decent performance for linear systems
- Classical Control is done in the “s-domain” (or frequency or Laplace Domain)
- However, PID is not the best choice for non-linear, time-varying, or multi-input, multi-output, or complex systems

2.1.2 Learning Outcomes

- Review the Laplace domain and its use in control systems
- Relate solutions of linear ODEs to concepts like, stability and performance
- Look into the concepts of proportional, integral, and derivative control

2.2 Slide 3 - Functions of a Complex Variable

Let $s = a + jb$ be a complex number with $j = \sqrt{-1}$. Notice we typically use j in controls since the Electrical Engineers took i for current.

We define the *rational polynomial* $G(s)$ as:

$$\begin{aligned} G(s) &= \frac{a_0 s^m + a_1 s^{m-1} + \dots + a_m}{s^n + b_1 s^{n-1} + \dots + b_n} \\ &= \frac{N(s)}{D(s)}, \text{ for } a_i, b_i \in \mathbb{R} \end{aligned}$$

When $m < n$, we say that $G(s)$ is *strictly proper*. When $m = n$, we say that $G(s)$ is *proper*. When $m > n$, we say that $G(s)$ is *improper*.

If we factor $G(s)$ into its roots, we can write:

$$G(s) = K \frac{(s-z_1)(s-z_2)\cdots(s-z_m)}{(s-p_1)(s-p_2)\cdots(s-p_n)}$$

where z_i are the *zeros* of $G(s)$ and p_i are the *poles* of $G(s)$.

Notice that the poles and zeros are complex numbers, and that the poles are the roots of the denominator polynomial $D(s)$ and the zeros are the roots of the numerator polynomial $N(s)$.

In set from $z_i = \{s \mid N(s) = 0\}$ and $p_i = \{s \mid D(s) = 0\}$. The *order* of $G(s)$ is the number of poles, n .

2.3 Slide 4 - Poles and Zeros Examples

$$G(s) = \frac{(s+2)(s+10)}{s(s+5)(s+15)^2}$$

Therefore,

$$z = \{-2, -10\}$$

$$p = \{0, -5, -15, -15\}$$

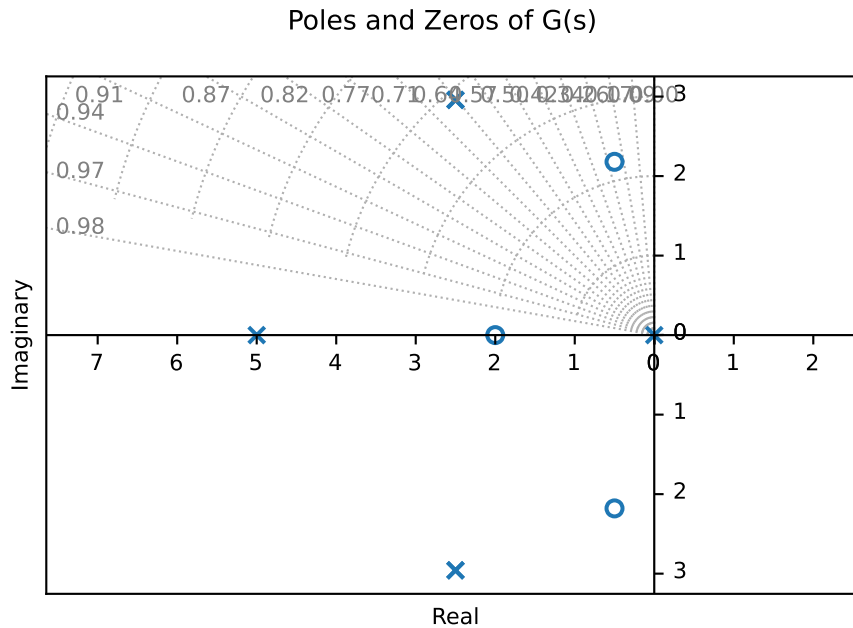
Another more complex example is:

$$G(s) = \frac{(s+2)(s^2+s+5)}{s(s+5)(s^2+5s+15)}$$

```
import numpy as np
import matplotlib.pyplot as plt
import control as ct

# Define the transfer function: G(s) = (s+2)(s^2+s+5) / (s(s+5)(s^2+5s+15))
num = np.polymul([1, 2], [1, 1, 5])
den = np.polymul([1, 0], np.polymul([1, 5], np.polymul([1, 5, 15], [1, 5, 15])))
G = ct.TransferFunction(num, den)

# Plot the poles and zeros
plt.figure()
ct.pole_zero_plot(G, title='Poles and Zeros of G(s)', grid=True)
plt.show()
```



2.4 Slide 5 - The Laplace Transform

The Laplace transform converts a time-domain signal into the complex frequency domain. For a causal function $f(t)$ (defined for $t \geq 0$) the unilateral Laplace transform is

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^{\infty} e^{-st} f(t) dt$$

,

where $s = \sigma + j\omega$ is a complex variable and the integral converges in the region of convergence (ROC).

Drawing of mapping from time domain to s-domain and back.

2.5 Slide 6 - Properties of the Laplace Transform

Key properties:

- Linearity: $\mathcal{L}\{af(t) + bg(t)\} = aF(s) + bG(s)$.
- Time shift: $\mathcal{L}\{f(t-a)u(t-a)\} = e^{-as}F(s)$.
- Frequency shift: $\mathcal{L}\{e^{at}f(t)\} = F(s-a)$.
- Differentiation (time): $\mathcal{L}\{f'(t)\} = sF(s) - f(0^+)$.

- Differentiation (time, higher order): $\mathcal{L}\{f^{(n)}(t)\} = s^n F(s) - s^{n-1}f(0^+) - \dots - f^{(n-1)}(0^+)$.
- Integration (time): $\mathcal{L}\{\int_0^t f(\tau) d\tau\} = \frac{1}{s}F(s)$.
- Integration (time, higher order): $\mathcal{L}\{\int_0^t \int_0^{\tau_1} \dots \int_0^{\tau_{n-1}} f(\tau_n) d\tau_n \dots d\tau_2 d\tau_1\} = \frac{1}{s^n}F(s)$.

Common transforms (causal signals):

- $\mathcal{L}\{1\} = \frac{1}{s}$
- $\mathcal{L}\{t\} = \frac{1}{s^2}$
- $\mathcal{L}\{e^{at}\} = \frac{1}{s-a}$
- $\mathcal{L}\{\sin(\omega t)\} = \frac{\omega}{s^2 + \omega^2}$
- $\mathcal{L}\{\cos(\omega t)\} = \frac{s}{s^2 + \omega^2}$

2.6 Slide 7 - Useful Theorems

Useful theorems:

- Initial value theorem: $f(0^+) = \lim_{s \rightarrow \infty} sF(s)$.
- Final value theorem: $\lim_{t \rightarrow \infty} f(t) = \lim_{s \rightarrow 0} sF(s)$, when the limits exist (i.e., poles of $sF(s)$ are in the left half-plane).

Quick example: $f(t) = e^{2t}u(t)$ has transform $F(s) = 1/(s-2)$ for $\text{Re}(s) > 2$.

The Laplace transform is fundamental in linear systems analysis because it turns linear differential equations into algebraic equations in s , simplifying solution and control-design tasks.

2.7 Slide 8 - Why Laplace?

One way to think about the Laplace transform is that it is a generalization of the Fourier transform. The Fourier transform is a special case of the Laplace transform where $s = j\omega$ and $\sigma = 0$. The Laplace transform allows us to analyze systems with exponential growth or decay, which is common in control systems.

The Laplace Transform is helpful because it transforms differential equations into algebraic equations, making it easier to analyze and design control systems.

If we can solve the algebraic equations in the Laplace domain, we can then use the inverse Laplace transform to find the solution in the time domain.

2.7.1 Example - Mass-Spring-Damper System (MSD)

Figure of MSD

The equation of motion for a mass-spring-damper system is given by:

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = F(t)$$

Let $\omega_n^2 = k/m$ where ω_n is the natural frequency, and $\frac{c}{m} = 2\zeta\omega_n$, so $\zeta = \frac{c}{2\sqrt{mk}}$ is the damping ratio, and $F = ku(t)$ is the force required to displace the mass some distance, $u(t)$ according to Hook's law. Finally, divide both sides by m to get:

$$\ddot{x}(t) + 2\zeta\omega_n\dot{x}(t) + \omega_n^2x(t) = \omega_n^2u(t)$$

Now, define the state vector as $\underline{x}(t) = \begin{bmatrix} x(t) \\ \dot{x}(t) \end{bmatrix}$, and we can write the system in state-space form as:

$$\dot{\underline{x}}(t) = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix} \underline{x}(t) + \begin{bmatrix} 0 \\ \omega_n^2 \end{bmatrix} u(t)$$

In more compact form, we can write this as:

$$\dot{\underline{x}}(t) = A\underline{x}(t) + B\underline{u}(t)$$

This is known as the state-space representation or state equation of the system, where A is the system matrix and B is the input matrix.

where $A = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}$ and $B = \begin{bmatrix} 0 \\ \omega_n^2 \end{bmatrix}$.

Assuming we measure the position of the mass, we can define the output as

$$y(t) = C\underline{x}(t)$$

where $C = [1 \ 0]$. This is known as the *output* or measurement equation of the system.

In control theory, we can generally let the output be a function of the input, i.e., $\underline{y}(t) = C\underline{x}(t) + D\underline{u}(t)$, where D is a *feed through* term. In this case, we have no *feed through*, so $D = 0$.

Now, we can take the Laplace transform of the state-space equations to get:

$$\mathcal{L}\{\dot{\underline{x}}(t)\} = \mathcal{L}\{A\underline{x}(t) + B\underline{u}(t)\}$$

$$s\underline{X}(s) - \underline{x}(0) = A\underline{X}(s) + B\underline{U}(s)$$

where $\underline{X}(s)$ is the Laplace transform of the state vector $\underline{x}(t)$, and $\underline{U}(s)$ is the Laplace transform of the input vector $\underline{u}(t)$.

Assuming $\underline{x}(0) = 0$, we can solve for $\underline{X}(s)$:

$$\underline{X}(s) = (sI - A)^{-1}B\underline{U}(s)$$

Finally, we can find the output in the Laplace domain:

$$\underline{Y}(s) = C\underline{X}(s) = C(sI - A)^{-1}B\underline{U}(s)$$

We define the *transfer function* of the system $G(s)$ as the ratio of the output to the input in the Laplace domain:

$$G(s) = \frac{\underline{Y}(s)}{\underline{U}(s)} = C(sI - A)^{-1}B$$

From before we notice,

$$G(s) = \frac{N(s)}{D(s)}$$

Therefore,

$$Y(s) = G(s)U(s) = C(sI - A)^{-1}BU(s) = \frac{N(s)}{D(s)}$$

2.8 Slide 9 - Another look at the Transfer Function

The transfer function is,

$$G(s) = \frac{N(s)}{D(s)}$$

Recall that for any matrix P , the inverse of P is given by,

$$P^{-1} = \frac{adj(P)}{det(P)}$$

where $adj(P)$ is the adjugate of P and $det(P)$ is the determinant of P .

The adjugate of a matrix is the conjugate transpose of the cofactor matrix.

This is the least efficient way to compute the inverse of a matrix, but it is useful for theoretical analysis.

Notice the adugate is a matrix of polynomials in s , and is $n \times n$ where n is the order of the system. The determinant is a polynomial in s of order n . Therefore, we can write,

$$(sI - A)^{-1} = \frac{adj(sI - A)}{det(sI - A)}$$

For the MSD system the output matrix is C is 1×2 and the input matrix is B is 2×1 . Therefore the denominator the transfer function is a scalar polynomial.

And it is always true that $det(sI - A) = D(s)$, the denominator of the transfer function. The numerator is a polynomial in s of order $n - 1$.

What is interesting here is that,

$$det(sI - A) = D(s) = \prod_{i=1}^n (s - p_i) = 0$$

where p_i are the poles of the transfer function. Therefore, the poles of the transfer function are the eigenvalues of the system matrix A .

$$\{Eigenvalues\ of\ A\} = \{Poles\ of\ the\ transfer\ function\ G(s)\}$$

2.9 Slide 10 - Step Response

The step response of a system is the output of the system when the input is a unit step function. The unit step function is defined as:

$$u_s(t) = \begin{cases} 0, & t < 0 \\ 1, & t \geq 0 \end{cases}$$

draw the step function

The Laplace transform of the unit step function is:

$$\mathcal{L}\{u_s(t)\} = \frac{1}{s}$$

Therefore, the step response of a system with transfer function $G(s)$ is given by:

$$Y(s) = G(s)U(s) = G(s)\frac{1}{s}$$

For the MSD system, we have:

$$Y(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \frac{1}{s}$$

From a variety of sources, we can find the inverse Laplace transform of this expression to get the step response in the time domain.

The step response of the MSD system is given by:

$$y(t) = 1 - \frac{1}{\sqrt{1-\zeta^2}} e^{-\zeta\omega_n t} \sin(\omega_d t + \phi)$$

where $\omega_d = \omega_n \sqrt{1-\zeta^2}$ is the damped natural frequency and $\phi = \arccos(\zeta)$ is the phase angle.

We can plot the plot the step response for $\zeta = 0.1, 0.3, 0.5, 0.7, 0.9$ and $\omega_n = 4$ rad/s.

```

import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt

# System parameters
zeta = [0.1, 0.3, 0.5, 0.7, 0.9]
wn = 4

# Create a figure
plt.figure(figsize=(10, 6))
t = np.linspace(0, 10, 1000)
for i, z in enumerate(zeta):
    # Transfer function coefficients
    num = [wn**2]
    den = [1, 2*z*wn, wn**2]

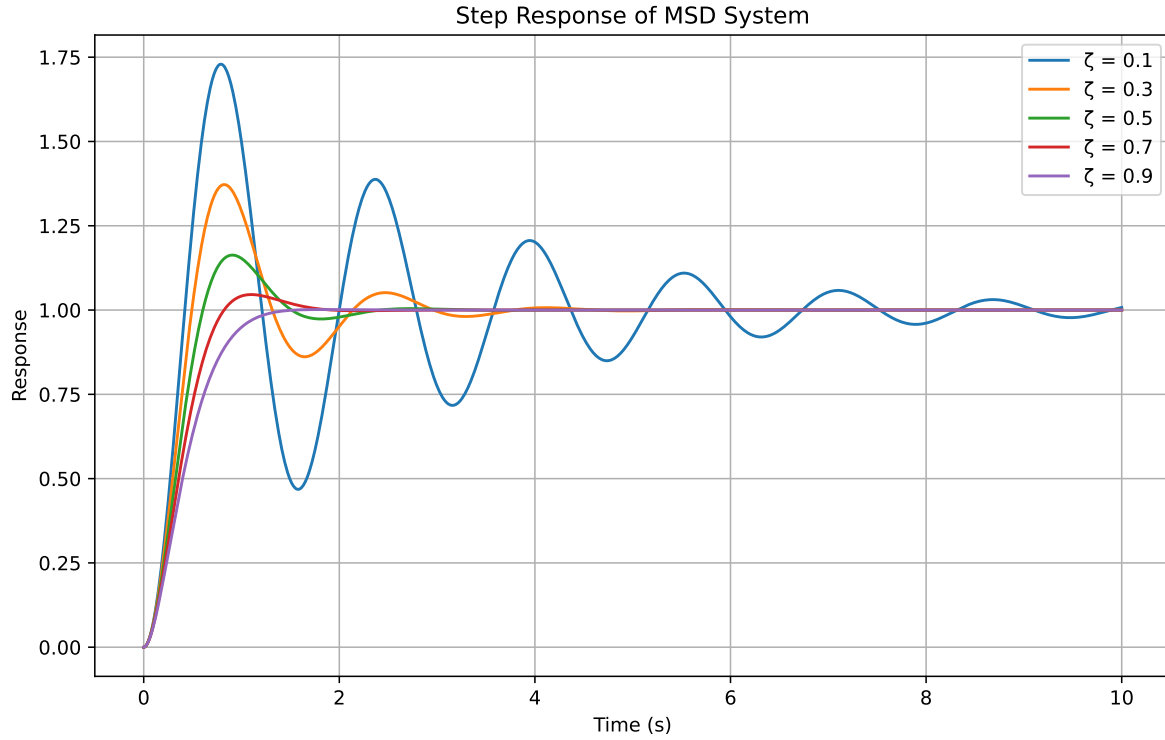
    sys = signal.TransferFunction(num, den)

    # Compute step response over our defined timeline
    t_out, y = signal.step(sys, T=t)

    # Plot step response
    plt.plot(t_out, y, label=f'z = {z}')

plt.xlabel('Time (s)')
plt.ylabel('Response')
plt.title('Step Response of MSD System')
plt.legend()
plt.grid(True)
plt.show()

```



In the case of the MSD system, we can find the poles of the transfer function by finding the roots of the characteristic polynomial $D(s) = s^2 + 2\zeta\omega_n s + \omega_n^2$. The poles are given by:

$$p_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$$

or,

$$p_{1,2} = -\zeta\omega_n \pm j\omega_d$$

where $\omega_d = \omega_n\sqrt{1-\zeta^2}$ is the damped natural frequency.

Finally, we can plot the poles of the MSD system for different values of ζ .

```
import numpy as np
import matplotlib.pyplot as plt

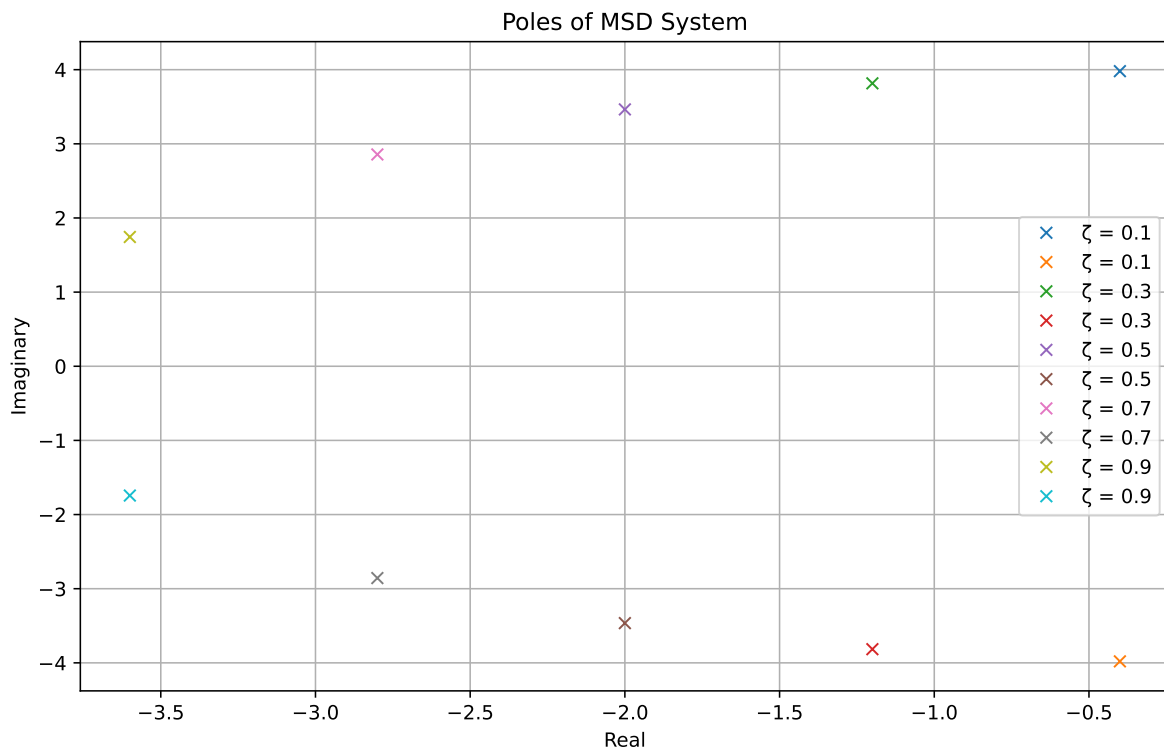
zeta = [0.1, 0.3, 0.5, 0.7, 0.9]
wn = 4

# Create a figure
```

```

plt.figure(figsize=(10, 6))
for z in zeta:
    # Calculate poles
    p1 = -z*wn + 1j*wn*np.sqrt(1-z**2)
    p2 = -z*wn - 1j*wn*np.sqrt(1-z**2)
    plt.plot(np.real(p1), np.imag(p1), 'x', label=f'  $\zeta = {z}$ ')
    plt.plot(np.real(p2), np.imag(p2), 'x', label=f'  $\zeta = {z}$ ')
plt.xlabel('Real')
plt.ylabel('Imaginary')
plt.title('Poles of MSD System')
plt.legend()
plt.grid(True)
plt.show()

```



If we plot the transfer function as a function of a single parameter, we can see how the poles move in the complex plane as we change the damping ratio ζ . This is known as a *root locus* plot. We can plot the root locus for any single variable.

What happens if the damping ratio is greater than 1? Or negative?

2.10 Slide 11 - Performance of the MSD Step Response

Plot the step response and call out the different performance metrics, such as rise time, settling time, overshoot, and steady-state error. Discuss how these metrics are affected by the damping ratio ζ and natural frequency ω_n .

Add a figure showing the step response with annotations for rise time, settling time, overshoot, and steady-state error.

2.11 Slide 12 - Performance Metrics

Performance:

Quantity	Formula
Rise Time	$t_r = \frac{\pi - \beta}{\omega_d}$
Peak Time	$t_p = \frac{\pi}{\omega_d}$
Overshoot	$M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$
Settling Time	$t_s \approx \frac{4}{\zeta\omega_n}$

Key point: ζ and ω_n tell us about performance and stability of the system.

If we can use feedback to effectively change the poles of the system, we can improve performance and stability.

2.12 Slide 13 - Performance and Stability with Feedback

2.12.1 Proportional Control

Draw a block diagram of a feedback control system with a plant $G(s)$ and a controller $C(s)$. With $C(s) = k_p$. Show how the closed-loop transfer function is given by:

$$Y(s) = G(s)U(s)$$

Point out the idea of *convolution* and the inverse Laplace Transform.

$$U(s) = C(s)E(s)$$

$$E(s) = U_r(s) - Y(s)$$

Therefore, we can write the closed-loop transfer function as:

$$Y(s) = G(s)C(s)E(s) = G(s)C(s)(U_r(s) - Y(s))$$

Solving for $Y(s)$, we get:

$$Y(s) = \frac{G(s)C(s)}{1 + G(s)C(s)} U_r(s)$$

Draw *closed loop* block diagram from $U_r(s)$ to $Y(s)$.

Now substitute $C(s) = k_p$ and $G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$ to get:

$$Y(s) = \frac{k_p \omega_n^2}{s^2 + 2\zeta\omega_n s + (1 + k_p)\omega_n^2} U_r(s)$$

Notice the poles and Eigenvalues of the closed-loop system are a function of the proportional gain k_p . Therefore, we can use feedback to change the poles of the system and improve performance and stability.

2.13 Slide 14 - Proportional Feedback

Let $\bar{\omega}_n^2 = (1 + k_p)\omega_n^2$ and $\bar{\zeta} = \frac{\zeta}{\sqrt{1+k_p}}$. Then we can write the closed-loop transfer function as:

$$G_c(s) = \frac{k_p \omega_n^2}{s^2 + 2\bar{\zeta}\bar{\omega}_n s + \bar{\omega}_n^2}$$

2.14 Slide 15 - Final Value Theorem

FVT: $\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} sY(s)$

Since the Laplace transform of the unit step function is $U(s) = 1/s$, we can use the final value theorem to write:

$$\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} sY(s) = \lim_{s \rightarrow 0} sG_c(s)U(s) = \lim_{s \rightarrow 0} G_c(s)$$

$$\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} G_c(s) = \lim_{s \rightarrow 0} \frac{k_p \omega_n^2}{s^2 + 2\bar{\zeta} \bar{\omega}_n s + \bar{\omega}_n^2} = \frac{k_p}{\bar{\omega}_n^2}$$

$$\lim_{t \rightarrow \infty} y(t) = \frac{k_p}{(1 + k_p)} \neq 1$$

2.15 Slide 16 - Proportional + Derivative Feedback

Now, since we have the closed loop Transfer Function, let's update the control law to a *PD* control law, where $C(s) = k_p + k_d s$.

Notice with this control law, $C(s)E(s) = k_p E(s) + k_d s E(s)$ and in the time domain, this is equivalent to $u(t) = k_p e(t) + k_d \dot{e}(t)$.

Then we can write the closed-loop transfer function as:

$$G_c(s) = \frac{(k_p + k_d s) \omega_n^2}{s^2 + (2\zeta \omega_n + k_d \omega_n^2) s + (1 + k_p) \omega_n^2}$$

Let $\bar{\omega}_n^2 = (1 + k_p) \omega_n^2$ and $\bar{\zeta} = \frac{2\zeta \omega_n + k_d \omega_n^2}{2\bar{\omega}_n}$, then we can write the closed-loop transfer function as:

$$G_c(s) = \frac{(k_p + k_d s) \omega_n^2}{s^2 + 2\bar{\zeta} \bar{\omega}_n s + \bar{\omega}_n^2}$$

So we can modify the closed-loop damping ratio and natural frequency by changing the proportional and derivative gains, k_p and k_d . We are also moving the poles around.

Notice we also added a zero which changes the transient response of the system. The zero is located at $s = -k_p/k_d$.

Finally, applying the final value theorem, we can see that the steady-state error is still non-zero:

$$\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} G_c(s) = \frac{k_p}{(1 + k_p)}$$

2.16 Slide 17 - Full PID Feedback

Let $C(s) = k_p + k_d s + \frac{k_i}{s}$, then we can write the closed-loop transfer function as:

$$G_c(s) = \frac{(k_p + k_d s + \frac{k_i}{s})\omega_n^2}{s^2 + (2\zeta\omega_n + k_d\omega_n^2)s + (1 + k_p)\omega_n^2 + k_i\omega_n^2}$$

or

$$G_c(s) = \frac{(k_d s^2 + k_p s + k_i)\omega_n^2}{s^3 + (2\zeta\omega_n + k_d\omega_n^2)s^2 + (1 + k_p)\omega_n^2 s + k_i\omega_n^2}$$

Notice we added two zeros and a pole. Where are the new poles and zeros located? How does this change the transient response of the system?

Plot the pzmap of the new closed-loop transfer function and discuss how the poles and zeros have changed.

But applying the Final Value Theorem, we can see that the steady-state error is now zero:

$$\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} G_c(s) = 1$$

There are many ways to tune a PID controller, but one of the most common methods is the Ziegler-Nichols method. This method involves finding the ultimate gain and ultimate period of the system, and then using these values to calculate the PID gains.

We can also use the root locus method to tune a PID controller. This method involves plotting the root locus of the system and then selecting the desired closed-loop poles based on the desired performance specifications.

Or we can use frequency response methods, such as Bode plots or Nyquist plots, to tune a PID controller. These methods involve analyzing the frequency response of the system and then selecting the PID gains based on the desired performance specifications.

Or we can use optimization methods, such as genetic algorithms or particle swarm optimization, to tune a PID controller. These methods involve defining a cost function based on the desired performance specifications and then using an optimization algorithm to find the PID gains that minimize the cost function.

3 System Modeling

3.1 Slide 2 - Motivation and Learning Outcomes

3.1.1 Developing a models is step *ZERO*

- Over the years several modeling techniques have been developed.
- We want to explore their roots
- Different methods have better (or worse) applications
- In the Aero/Mechanical world we are generally concerned with:
 - Dynamics/Mechanics
 - Vibrations
 - ??

3.2 Slide 3 - The Heritage of Mechanics

Free body diagrams and Newton's description of motion. The Principia: mathematical principles of natural philosophy

$$\vec{F} = m\vec{a} = \frac{d}{dt}(m\vec{v})$$

For the MSD system,

$$m\ddot{q} + c\dot{q} + kq = f(t)$$

3.3 Slide 4 - State, Time, and Space

We plot the response of the MSD system explicitly in time.

We can also plot the response in state space, where we plot the position and velocity of the system as a function of time. This allows us to visualize the dynamics of the system in a different way.

We can represent a system of higher order differential equations as a system of first order differential equations. This is the basis of the state space representation of a system.

This approach lends itself to numerical solving methods like Runge-Kutta and the tools of vector spaces and Real/Complex analysis.

i Note

Do IC plot and Phase Plane Matlab example for the MSD system.

- State Space gives a sense of flow and the vector field of the system for any initial conditions.
- State Space gives a sense of the stability of the system, and how it will respond to perturbations.

3.4 Slide 5 - The Heritage of Electrical Engineering

Fundamental to this approach is the idea of input and output of a system. Systems can be reduced to subsystems, etc., and analyzed as a whole. This is the basis of the block diagram approach to modeling systems.

3.5 Slide 6 - Step and Frequency Response

We've talked about the step response from a performance perspective, but we can also look at the step response from a modeling perspective. The step response is a fundamental tool for understanding the dynamics of a system.

The frequency response is another fundamental tool for understanding the dynamics of a system. The frequency response tells us how the system responds to different frequencies of input, and can be used to analyze the stability and performance of the system.

Mostly we want to understand how the magnitude and phase of the output changes with frequency. This is the basis of Bode plots, which are a fundamental tool for analyzing the frequency response of a system.

i Note

Do step response and Bode plot Matlab example for the MSD system.

3.6 Slide 7 - The Control View - State Space

$$\frac{dx}{dt} = f(x, u), \quad y = h(x, u)$$

x is the *state* vector u is the *input* vector y is the *output* vector

f is the *state function* h is the *output function*

These functions describe how the state of the system evolves over time, and how the output of the system is related to the state and input.

With these functions and these views, it is natural to ask questions like:

- Does the state converge? (Stability)
- What states can the system reach given the proper choice of $u(t)$? (controllability or reachability)
- If we don't measure the whole state, can we reconstruct the state? (observability)
- Notice the answers to these questions should not depend on the system we are working with, but on properties of the model

3.7 Slide 8 - Summary

4 State Space Modeling

4.1 Slide 2 - Motivation and Learning Outcomes

4.1.1 The Fundamental Abstraction

- Dynamic System Theory as a branch of mathematics gives us a foundational starting point
- We will rely on a variety of math tools, but primarily Linear Algebra and Analysis

4.2 Slide 3 - Start with ODEs

Let:

$$\begin{aligned}\dot{\underline{x}} &= \underline{f}(\underline{x}, \underline{u}, t) \\ y &= \underline{h}(\underline{x}, \underline{u}, t)\end{aligned}$$

Where $\underline{x} \in \mathbb{R}^n$ is the state vector, $\underline{u} \in \mathbb{R}^q$ is the input vector, and $y \in \mathbb{R}^p$ is the output/measurement vector. The function $\underline{f}$ describes how the state evolves over time, while $\underline{h}$ describes how the output is generated from the state and input.

and $\underline{f} : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^n$ and $\underline{h} : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^q$ are vector-valued functions that describe the dynamics of the system.

This is the State Space Model of a system.

4.3 Slide 4 - For LTI systems

We have the LTI State Space Model:

$$\begin{aligned}\dot{\underline{x}} &= A\underline{x} + B\underline{u} \\ y &= C\underline{x} + D\underline{u}\end{aligned}$$

Where $A \in \mathbb{R}^{n \times n}$ is the state or dynamics matrix, $B \in \mathbb{R}^{n \times q}$ is the input or control matrix, $C \in \mathbb{R}^{p \times n}$ is the output or measurement matrix, and $D \in \mathbb{R}^{p \times q}$ is the feed through or direct matrix.

The matrices A , B , C , and D are constant for LTI systems.

4.4 Slide 7 - Example - The Inverted Pendulum

i Note

Draw a figure of the inverted pendulum. Theta is defined positive counter-clockwise from vertical.

$$J_t \ddot{\theta} + \gamma \dot{\theta} - mgl \sin(\theta) = u$$

Where $J_t = J + ml^2$ is the total moment of inertia of the pendulum about the pivot point, γ is the damping coefficient, m is the mass of the pendulum, g is the acceleration due to gravity, l is the length of the pendulum, and u is the control input (torque applied at the pivot point).

4.5 Slide 8 - State Space Model of the Inverted Pendulum

Let $\underline{x} = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix}$, then we can write the state space model as:

$$\dot{\underline{x}} = \begin{bmatrix} \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{\theta} \\ -\frac{\gamma}{J_t} \dot{\theta} + \frac{mgl}{J_t} \sin(\theta) + \frac{u}{J_t} \end{bmatrix}$$

Where $\underline{f}(\underline{x}, \underline{u}) = \begin{bmatrix} \dot{\theta} \\ -\frac{\gamma}{J_t} \dot{\theta} + \frac{mgl}{J_t} \sin(\theta) + \frac{u}{J_t} \end{bmatrix}$.

Now say we measure the horizontal displacement d . That gives,

$$\underline{y} = l \sin(\theta)$$

Where $\underline{h}(\underline{x}, \underline{u}) = l \sin(\theta)$.

4.6 Slide 9 - Linearization of the Inverted Pendulum

When $\theta \approx 0$, we can linearize the system about the equilibrium point $\theta = 0$ to obtain the LTI state space model:

$$\dot{\underline{x}} = \begin{bmatrix} 0 & 1 \\ \frac{mgl}{J_t} & -\frac{\gamma}{J_t} \end{bmatrix} \underline{x} + \begin{bmatrix} 0 \\ \frac{1}{J_t} \end{bmatrix} u$$

$$\underline{y} = \begin{bmatrix} l & 0 \end{bmatrix} \underline{x}$$

Where $A = \begin{bmatrix} 0 & 1 \\ \frac{mgl}{J_t} & -\frac{\gamma}{J_t} \end{bmatrix}$ and $B = \begin{bmatrix} 0 \\ \frac{1}{J_t} \end{bmatrix}$ and $C = \begin{bmatrix} l & 0 \end{bmatrix}$ and $D = 0$.

4.7 Slide 10 - Simulink Example of the Inverted Pendulum

4.8 Slide 11 - Block Diagrams Overview

4.9 Slide 12 - Summary

4.9.1 Developing a model is step **ZERO** in the design of a control system.

- Modeling is a key step in developing a control system analysis
 - Without the model, we have no place to start
- The State Space approach is generally considered a good place to start
- We will use a variety of modeling tools
 - Math, linear analysis, ODEs,
 - Block diagrams
 - Matlab and Simulink
 - Python *...
- “All models are wrong, but some are useful.” – George E. P. Box

5 Dynamic Behavior - Stability

5.1 Motivation

- Understanding Dynamic Systems
 - There are several ways of looking at and analyzing dynamic systems
 - Each approach reveals different behaviors
 - We will look at:
 - * Existence and Uniqueness of solutions
 - * Phase portraits, equilibrium and limit cycles
 - * Stability

5.2 Slide 3 - Solving

Recall the general form of a dynamic system: $\dot{\underline{x}} = f(\underline{x}, \underline{u}, t)$ and $\underline{y} = h(\underline{x}, \underline{u}, t)$

Let $\underline{u} = \alpha \underline{x}$ i.e. a function of the state which is a common case in control systems.

Definition: $\underline{x}(t)$ is a solution to the state equation with initial condition $\underline{x}(0) = \underline{x}_0$ for all $t \in \mathbb{R}$ if $\underline{x}(t_0) = \underline{x}_0$ and $\dot{\underline{x}}(t) = f(\underline{x}, \alpha(\underline{x})) = \underline{F}(\underline{x})$ for all $t_0 \leq t \leq t_1$. In this case, $\{t_0, t_1\}$ can be open or closed at t_1 .

5.3 Slide 4 - Examples - Finite Escape Time

$\dot{x} = x^2$ with $x(0) = 1$ has a solution $x(t) = \frac{1}{1-t}$ which is valid for $t \in [0, 1)$ and has a finite escape time at $t = 1$.

In this case, $x(t) \rightarrow \infty$ as $t \rightarrow 1$.

5.4 Slide 5 - Non-uniqueness of Solutions

$\dot{x} = 2\sqrt{x}$ with $x(0) = 0$ has an infinite number of solutions.

$$x(t) = \begin{cases} 0, & t \in [0, a] \\ (t-a)^2, & t \in (a, \infty) \end{cases}$$

Where a is any positive number.

For verification, $\frac{dx}{dt}(t-a)^2 = 2(t-a) = 2\sqrt{(t-a)^2} = 2\sqrt{x}$ for $t > a$ and $\dot{x} = 0 = \sqrt{0} = \sqrt{x}$ for $t \leq a$.

5.5 Slide 6 - Lipschitz Continuity

Definition: A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is Lipschitz continuous if there exists a constant $c > 0$ such that for all $\underline{x}, \underline{y} \in \mathbb{R}^n$,

$$\|F(\underline{x}) - F(\underline{y})\| \leq c\|\underline{x} - \underline{y}\|$$

- If F is Lipschitz continuous, then the solution to $\dot{\underline{x}} = F(\underline{x})$ is unique.
- If F is continuously differentiable, then F is Lipschitz continuous
- If ∇F is uniformly bounded, then F is Lipschitz continuous

5.6 Slide 7 - Examples of Lipschitz Continuity

Note

Draw a figure in state space showing two points $\underline{x}$ and $\underline{y}$ and the corresponding vectors $F(\underline{x})$ and $F(\underline{y})$. Show the distance between the points and the distance between the vectors, illustrating the Lipschitz condition.

In the case of the example $\dot{x} = x^2$, $F(x) = x^2$ is *not* Lipschitz continuous since $\frac{dF}{dx} = 2x$ is unbounded as $x \rightarrow \infty$.

In the case of the example $\dot{x} = 2\sqrt{x}$, $F(x) = 2\sqrt{x}$ is not Lipschitz continuous at $x = 0$ since $\frac{dF}{dx} = \frac{1}{\sqrt{x}}$ is unbounded as $x \rightarrow 0$.

More from Wikipedia on [Uniform boundedness](#)

5.7 Slide 8 - Qualitative Analysis - Phase Portraits

Notice $\dot{\underline{x}} = F(\underline{x})$ is a *vector field* in $\mathbb{R}^n$ and can be visualized as a *phase portrait* in $\mathbb{R}^n$.

If we start at a point $\underline{x}_0$ in the phase portrait, we can follow the vector field to see how the state evolves over time. This is a *trajectory* in the phase portrait.

The phase portrait is a collection of trajectories in the state space.

5.8 Slide 9 - Equilibrium Points

Definition: The point in the state-space graph, $\underline{x}_e$ is an *equilibrium point* of the dynamic system $\dot{\underline{x}} = F(\underline{x})$ if $F(\underline{x}_e) = 0$ for all $t \geq t_0$.

Note

Example: MSD - for the system $\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = 0$, the equilibrium point is at $x = 0$ and $\dot{x} = 0$.

Since $\dot{\underline{x}} = \begin{bmatrix} \dot{x} \\ \ddot{x} \end{bmatrix} = \begin{bmatrix} \dot{x} \\ -2\zeta\omega_n\dot{x} - \omega_n^2x \end{bmatrix}$, we have $F(\underline{x}) = 0$ when $\dot{x} = 0$ and $x = 0$. Therefore, the equilibrium point is at $\underline{x}_e = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

5.9 Slide 10 - Examples

Note

Use Matlab pplane to plot the following:

- $\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = 0$ for various values of ζ and ω_n .
- $\ddot{x} + \sin x = 0$
- $\ddot{x} - \sin x = 0$
- $\ddot{x} + \dot{x} + \sin x = 0$
- $\ddot{x} + (1 - x^2)\dot{x} + x = 0$ (Van der Pol Oscillator)

Note

Notice: - The single equilibrium point at the origin for the MSD system. - The infinite number of equilibrium points for the pendulum system. - The limit cycle for the Van der

Pol oscillator.

5.10 Slide 11 - Summary

There is always more:

- For linear systems, things are straight forward
 - Single isolated equilibrium points that can be transformed to any coordinate frame
- For non-linear systems, not so much
- We will develop tools to analyze non-linear systems and see how they apply to the subset of linear systems

6 Linear Systems

6.1 Modal Example - 2 MSD system

```
import numpy as np
import scipy.linalg as la
import matplotlib.pyplot as plt

# Mass Matrix
m_1 = 1.0
m_2 = 1.0
M = np.array([[m_1, 0], [0, m_2]])

# Stiffness Matrix
k_1 = 1
k_2 = 1
K = np.array([[k_1 + k_2, -k_2], [-k_2, k_1 + k_2]])

# Step 1 - Compute the Modal Matrix
P = la.sqrtm(la.inv(M)) @ K @ la.sqrtm(la.inv(M))

# Step 2 - Compute the Eigenvalues and Eigenvectors of P
Lambda, Psi_bar = la.eig(P)

# Step 3 - Compute the Mode Shapes
Psi_0 = la.sqrtm(la.inv(M)) @ Psi_bar

print("Eigenvalues (Mode Frequencies):")
print(Lambda)

print("Mode Shapes:")
print(Psi_0)
```

```
Eigenvalues (Mode Frequencies):
[3.+0.j 1.+0.j]
```

Mode Shapes:

```
[[ 0.70710678  0.70710678]
 [-0.70710678  0.70710678]]
```

6.1.1 Modal Decomposition Process

1. $P = M_0^{\frac{1}{2}} K M_0^{\frac{1}{2}}$
2. Find the Eigenvalues and Eigenvectors of P , $\{\bar{\Psi}_0, \bar{\Lambda}_0\} = \mathbf{eig}(P)$.
 - Mode frequencies are the Eigenvalues of P in $\bar{\Lambda}_0$.
3. Find the Mode Shapes, $\Psi_0 = M_0^{\frac{1}{2}} \bar{\Psi}_0$

7 Root Locus

7.1 Learning Objectives

- Sketch root loci

8 Frequency Response

8.1 Learning Objectives

- Bode plots
- Nyquist criterion

9 PID Design

9.1 Learning Objectives

- Tune P, PI, PD, PID controllers

A Matrices and Vector Spaces: A Compact Reference

Structure, transformations, decompositions, and numerical sensitivity

A.1 The unifying idea

A [vector space](#) is a collection of objects that can be added and scaled while satisfying the familiar algebraic rules. A [matrix](#) is a rectangular array of entries, but its more important role in linear algebra is to represent a **linear transformation** after bases have been chosen. This viewpoint connects the topics on this page:

vectors $\longrightarrow$ linear combinations and subspaces $\longrightarrow$ matrix transformations $\longrightarrow$ structure and numerical behavior

For a matrix $A \in \mathbb{F}^{m \times n}$, the associated linear map is

$$A : \mathbb{F}^n \rightarrow \mathbb{F}^m, \quad x \mapsto Ax,$$

where the field $\mathbb{F}$ is commonly $\mathbb{R}$ or $\mathbb{C}$.



Reading matrices as transformations

The columns of A are the images of the coordinate basis vectors. Consequently, Ax is a linear combination of the columns of A , with the entries of x supplying the coefficients.

A.2 Vector spaces, span, and dimension

A vector space V over a field $\mathbb{F}$ is closed under vector addition and scalar multiplication and obeys the standard associativity, commutativity, identity, inverse, and distributive laws. Its elements need not be arrows: vectors may be polynomials, functions, signals, matrices, or state trajectories. See the linked [definition and notation](#).

Given vectors $v_1, \dots, v_k \in V$, their [linear span](#) is the set of all linear combinations

$$\text{span}\{v_1, \dots, v_k\} = \left\{ \sum_{i=1}^k \alpha_i v_i : \alpha_i \in \mathbb{F} \right\}.$$

The span is the smallest subspace containing the listed vectors. A **basis** is a linearly independent set that spans the entire space, and the number of vectors in any finite basis is the **dimension** of the space.

For a matrix $A = [a_1 \ \cdots \ a_n]$, the column space is

$$\mathcal{C}(A) = \text{span}\{a_1, \dots, a_n\}.$$

This makes span the bridge from a list of matrix columns to the subspace of outputs that the transformation can produce.

A.3 Matrices and their basic operations

An $m \times n$ matrix has m rows, n columns, and entries a_{ij} :

$$A = [a_{ij}] = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}.$$

The basic operations summarized in [addition, scalar multiplication, and transposition](#) are:

Operation	Definition	Compatibility
Addition	$(A + B)_{ij} = a_{ij} + b_{ij}$	A and B have the same size
Scalar multiplication	$(\alpha A)_{ij} = \alpha a_{ij}$	$\alpha \in \mathbb{F}$
Transpose	$(A^T)_{ij} = a_{ji}$	$A \in \mathbb{F}^{m \times n}$ gives $A^T \in \mathbb{F}^{n \times m}$
Matrix multiplication	$(AB)_{ij} = \sum_k a_{ik} b_{kj}$	inner dimensions agree

Matrix multiplication represents composition of linear maps. It is associative and distributive but generally **not commutative**: AB need not equal BA .

For complex matrices, the [conjugate transpose](#) is

$$A^* = \overline{A}^T, \quad (A^*)_{ij} = \overline{a_{ji}}.$$

It replaces the ordinary transpose in complex inner-product calculations. A matrix satisfying $A^* = A$ is **Hermitian**; over the real numbers, this reduces to symmetry, $A^T = A$.

A.3.1 Block matrices

A [block matrix](#) partitions a large matrix into smaller matrices:

$$M = \begin{bmatrix} A & B \\ C & D \end{bmatrix}.$$

Provided the block dimensions are compatible, many ordinary matrix operations can be carried out blockwise. Block notation exposes system structure and is especially useful for coupled state-space models, partitioned variables, Schur complements, and structured elimination.

A.4 Determinant, rank, invertibility, and degeneracy

These ideas ask related but distinct questions about a transformation.

Concept	Main question	Key interpretation
Determinant	How does a square map scale oriented volume?	$\det(A) = 0$ means some dimension is collapsed
Rank	How many independent output directions remain?	$\text{rank}(A) = \dim \mathcal{C}(A)$
Invertibility	Can every output be uniquely mapped back to an input?	$A^{-1}A = AA^{-1} = I$
Degeneracy	Has a normally distinct or full-dimensional case collapsed?	Often appears as zero determinant, deficient rank, or repeated structure

For a square matrix $A \in \mathbb{F}^{n \times n}$, the following statements are equivalent:

$$\begin{aligned}
A \text{ is invertible} &\iff \det(A) \neq 0 \\
&\iff \text{rank}(A) = n \\
&\iff \text{null}(A) = \{0\} \\
&\iff Ax = b \text{ has a unique solution for every } b \\
&\iff 0 \text{ is not an eigenvalue of } A.
\end{aligned}$$

The determinant is multiplicative, $\det(AB) = \det(A)\det(B)$, and geometrically gives the signed factor by which a real linear transformation scales volume. Rank is the dimension of either the column space or the row space; the equality of row rank and column rank is fundamental. Elementary row reduction is a common way to compute rank.

“Degenerate” is context dependent. A singular matrix, a zero-volume transformation, a rank-deficient set of constraints, or a quadratic form with a nontrivial null direction may all be described as degenerate. The useful question is therefore: **which expected dimension, distinction, or independence has been lost?**

A.5 Trace, similarity, and diagonalization

The [trace](#) of a square matrix is the sum of its diagonal entries:

$$\text{tr}(A) = \sum_{i=1}^n a_{ii}.$$

It satisfies

$$\text{tr}(A + B) = \text{tr}(A) + \text{tr}(B), \quad \text{tr}(AB) = \text{tr}(BA),$$

and equals the sum of the eigenvalues, counted with algebraic multiplicity.

Two square matrices are [similar](#) when

$$B = P^{-1}AP$$

for some invertible P . Similar matrices are different coordinate representations of the same linear transformation. They share basis-independent properties including determinant, trace, characteristic polynomial, eigenvalues, and rank.

A matrix is [diagonalizable](#) if it is similar to a diagonal matrix:

$$A = PDP^{-1}.$$

The columns of P are then a basis of eigenvectors, and D contains the corresponding eigenvalues. Diagonal form makes powers, exponentials, and linear dynamical systems easier to analyze:

$$A^k = PD^kP^{-1}, \quad e^{At} = Pe^{Dt}P^{-1}.$$

Not every matrix is diagonalizable. A matrix is diagonalizable precisely when it has enough linearly independent eigenvectors to form a basis.

i Similarity is not the same as equality

A and $B = P^{-1}AP$ generally have different entries. They describe the same transformation in different bases, so their coordinate-dependent appearance changes while their intrinsic linear-algebraic properties remain.

A.6 Inner products, outer products, and norms

An **inner product** assigns a scalar to a pair of vectors and generalizes the dot product. In $\mathbb{R}^n$ and $\mathbb{C}^n$,

$$\langle x, y \rangle = x^*y.$$

It defines length and angle:

$$\|x\|_2 = \sqrt{\langle x, x \rangle}, \quad \cos \theta = \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}$$

for nonzero real vectors. The **Cauchy–Schwarz inequality**,

$$|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2,$$

guarantees that the normalized inner product behaves like a cosine and underpins the triangle inequality for the induced norm.

The **outer product** reverses the direction of the construction: two column vectors produce a matrix,

$$xy^* = \begin{bmatrix} x_1 \overline{y_1} & \cdots & x_1 \overline{y_n} \\ \vdots & \ddots & \vdots \\ x_m \overline{y_1} & \cdots & x_m \overline{y_n} \end{bmatrix}.$$

When x and y are nonzero, xy^* has rank one. Outer products therefore serve as elementary building blocks for low-rank matrices and decompositions.

A [norm](#) measures magnitude. It must satisfy positivity, absolute homogeneity, and the triangle inequality. Common vector norms are

$$\|x\|_1 = \sum_i |x_i|, \quad \|x\|_2 = \sqrt{\sum_i |x_i|^2}, \quad \|x\|_\infty = \max_i |x_i|.$$

A [matrix norm](#) measures the size of a matrix. An induced norm is defined by the maximum amplification of a chosen vector norm:

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

It therefore satisfies the consistency bound $\|Ax\| \leq \|A\|\|x\|$. For the Euclidean norm, the induced matrix norm is the largest singular value: $\|A\|_2 = \sigma_{\max}(A)$.

A.7 Singular value decomposition and conditioning

Every real or complex $m \times n$ matrix has a [singular value decomposition](#):

$$A = U\Sigma V^*,$$

where U and V are unitary (orthogonal in the real case) and Σ is diagonal apart from possible extra zero rows or columns. Its nonnegative diagonal entries

$$\sigma_1 \geq \sigma_2 \geq \dots \geq 0$$

are the singular values. Geometrically, V^* rotates or reflects the input coordinates, Σ stretches along orthogonal directions, and U rotates or reflects the result.

The SVD unifies several earlier concepts:

$$\begin{aligned} \text{rank}(A) &= \#\{i : \sigma_i > 0\}, \\ \|A\|_2 &= \sigma_1, \\ \|A\|_F &= \sqrt{\sum_i \sigma_i^2}, \\ A^\dagger &= V\Sigma^\dagger U^*. \end{aligned}$$

Truncating the SVD after r singular values produces a best rank- r approximation in the spectral and Frobenius norms.

The [condition number](#) describes sensitivity: how much relative error in inputs or data may be amplified in the output. For an invertible matrix,

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

In the 2-norm,

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

A value near 1 indicates well-balanced scaling. A large value indicates a direction that is strongly compressed relative to another, so solving $Ax = b$ may be sensitive to perturbations. A singular matrix has infinite condition number under the usual convention.

 Invertible does not mean numerically safe

A matrix can have a nonzero determinant and still be nearly singular. Exact invertibility is an algebraic yes-or-no property; conditioning measures how reliable computations involving that inverse are in finite precision or noisy data.

A.8 Quadratic forms and definite matrices

A [quadratic form](#) associated with a real symmetric matrix A is

$$q(x) = x^T Ax.$$

For complex vectors, the corresponding Hermitian form is $x^* Ax$. The sign of this scalar leads to the classification of a [definite matrix](#):

Classification	Requirement for every $x \neq 0$	Eigenvalues of Hermitian A
Positive definite	$x^* Ax > 0$	all positive
Positive semidefinite	$x^* Ax \geq 0$	all nonnegative
Negative definite	$x^* Ax < 0$	all negative
Negative semidefinite	$x^* Ax \leq 0$	all nonpositive
Indefinite	takes both positive and negative values	both positive and negative

For a real symmetric or complex Hermitian matrix, definiteness can be checked using eigenvalues; positive definiteness can also be tested with Sylvester's criterion or successful Cholesky factorization. Positive-definite matrices define ellipsoidal level sets and appear in energy functions, least squares, optimization, covariance models, and Lyapunov stability analysis.

Orthogonal or unitary diagonalization makes the geometry explicit:

$$A = Q\Lambda Q^*, \quad x^*Ax = (Q^*x)^*\Lambda(Q^*x) = \sum_i \lambda_i |z_i|^2.$$

Thus the eigenvalue signs determine the sign of the quadratic form in every direction.

A.9 How the concepts fit together

The main logical connections are:

1. A basis converts an abstract linear transformation into a matrix.
2. Span and rank describe which output directions the matrix can reach.
3. Full rank, nonzero determinant, and invertibility coincide for square matrices.
4. Similarity changes coordinates while preserving intrinsic properties.
5. Diagonalization uses an eigenvector basis when such a basis exists.
6. Inner products provide geometry; norms measure size; outer products build rank-one maps.
7. The SVD works for every matrix and exposes rank, amplification, approximation, and pseudoinversion.
8. The condition number turns singular-value imbalance into a measure of numerical sensitivity.
9. Quadratic forms and definiteness connect matrix structure to geometry, energy, convexity, and stability.

! Compact mental model

Use **rank and determinant** to ask whether information is lost; **similarity and diagonalization** to find useful coordinates; **inner products and norms** to measure geometry and size; **SVD and condition number** to understand numerical behavior; and **quadratic forms** to study direction-dependent sign or energy.

A.10 Source links

- [Matrix: overview and definition](#)

- Matrix operations
- Conjugate transpose
- Determinant
- Rank
- Invertible matrix
- Degeneracy
- Block matrix
- Trace
- Vector space
- Linear span
- Matrix similarity
- Diagonalizable matrix
- Inner product space
- Cauchy–Schwarz inequality
- Outer product
- Norm
- Matrix norm
- Condition number
- Singular value decomposition
- Quadratic form
- Definite matrix